Family Relationship Inference System Using Python

Dinanath Padhya*, Krishna Acharya†
*THA078BEI010 †THA078BEI020
Department of Electronics, Communication and Information Engineering
Thapathali Campus, IOE, Tribhuvan University, Nepal

***Abstract*—This report presents a relationship inference engine using Python. The task includes adding family facts such as parent-child relations and allowing queries and listing of derived relationships like siblings, grandparents, uncles, and nieces. A rule-based approach using sets and tuples is employed to store known facts and compute additional relations via helper functions. The system supports various commands such as 'add', 'list', 'query', and 'show', with the aim of interpreting basic kinship from minimal input. This project not only tests the ability to manage simple data but also reinforces logical deduction based on established relationships. Through incremental development and structured logic, the program simulates a reasoning model that mirrors how genealogical relationships work in real life.**

I. Introduction

Understanding and representing family relationships in a formalized way is a foundational problem in logic programming. The lab is focused on implementing such a system using Python sets and tuples. Sets are used to ensure no duplicate relationships are stored, while tuples encode relationships such as (relation, person1, person2).

This lab implements a system that supports adding facts and inferring more complex relationships. For instance, if a parent relationship is defined, child and grandparent relations can be inferred using transitive rules. The implementation is command-driven, enabling operations such as querying specific relationships and listing all relatives of a given type.

Representing and processing relationships is one of the basic yet powerful tasks in computing, often used in the design of knowledge-based systems, simulations, and even in database architecture. Modeling such logic with basic programming constructs deepens understanding of the fundamentals of computer science, particularly in managing sets, tuple data structures, and algorithmic logic.

II. Objectives

- To implement a basic fact storage system using Python sets.
- To infer complex relationships from basic facts.
- To enable queries and listings of supported relationships.
- To ensure the system works for both direct and indirect inferences.

III. Supported Features

The following features are implemented in the system:

- Add parent or child relationships.
- Automatically infer reciprocal relationships.
- List relations such as parent, child, sibling, grandparent, grandchild, uncle/aunt, and nephew/niece.
- Query specific relationships between any two persons.
- View all stored facts.
- Dynamic computation of indirect relations based on existing known pairs.
- Prevention of redundant entries using set semantics.

IV. Methodology

The core implementation revolves around a Python class named `FamilyRelations`. Facts are stored in a set as tuples in the form (relation, person1, person2). If a user adds a 'child' relation, the reciprocal 'parent' relation is automatically added. Each relationship is treated as a directed triple.

A. Adding Facts

The `add_fact` method is used to add either a parent or child relation. If a child is added, the parent relation is inferred. Adding facts is fundamental to expanding the knowledge base of the program. By using consistent logic, even a small number of input relationships can yield a wide network of inferences.

B. Retrieving Relationships

Helper methods such as `get_children`, `get_parents`, `get_siblings`, `get_grandparents`, and others are used to retrieve relations from the stored facts. For example, siblings are identified by finding people with shared parents. Grandparents are found by tracing back two parent links.

C. Inference

Indirect relationships such as sibling or uncle are inferred using shared parents or sibling's children respectively. For example, siblings are identified by checking who shares the same parent. Grandchildren are derived from the children of a person's children. Uncles and aunts are derived by checking the siblings of a person's parents, and nieces or nephews from the children of a person's siblings.

D. Query and List Commands

`query(relation, person1, person2)`: Returns True if the specified relation holds between the two.

`list_relation(relation, person)`: Lists all persons having the specified relation with the input person.

These commands help to verify and explore the relationship map interactively and ensure the model behaves as expected.

E. Python Code Snippet

```
def get_children(self, person):
return {child for (rel, p, child) in
    self.facts
if rel == 'parent' and p == person}
```

Listing 1. Function to Get Children of a Person

Other functions are structured similarly, iterating over the fact set and applying logic to filter the tuples based on the given relation and involved persons.

V. Testing and Evaluation

Various relationships were added manually using the system and complex relations were correctly inferred. Some examples include:

- Adding: parent Alice Bob
- Adding: parent Alice Charles
- Adding: parent Bob David
- Adding: parent Charles Emma
- Querying: sibling Bob Charles → Yes
- Querying: grandparent Alice Emma → Yes (via Charles)
- Listing: uncle Charles David → True

A sample of the program output when run:

```
All stored facts:
Parent: alice -> bob
Parent: alice -> charles
Parent: bob -> david
Parent: charles -> emma

List sibling bob: charles
List grandparent david: alice
Query parent alice bob: Yes
Query sibling bob charles: Yes
Query uncle charles david: Yes
```

VI. Discussion

The system accurately deduces complex relationships based on limited input. The use of sets eliminates duplicates and simplifies data management. Each command is stateless and derives results dynamically from existing facts.

Scenarios such as adopted relationships, step-siblings, or polygamous structures are not supported directly but can be added with minimal code changes by redefining the relationship logic.

The program encourages thinking in logical chains and understanding relational transitivity. Even though it is simple, the implications of its logic-based foundation are significant. Such a model can be extended toward decision support systems or as part of a broader genealogy software.

VII. Conclusion

The experiment successfully implemented a basic relationship engine that can store and infer family relationships using only tuples and sets. The logic for indirect inference like siblings and grandparents was clearly verified. The approach ensures extensibility and simplicity by using fundamental Python data structures. This lab reinforces concepts in set theory, transitive relationships, and functional programming using Python.

The current structure is efficient for small to moderate datasets. Future versions may use indexing or graph data structures for faster querying in large-scale scenarios. Additional enhancements may include support for deleting facts, exporting relationships, and generating visual family trees from the facts.